COMPONENT BASED SOFTWARE ENGINEERING

Basic concepts

INTRODUCTION

- A central idea in **Component-Based Software Engineering (CBSE)** is that large software systems can be constructed by combining smaller, reusable software components.
- For components to work together successfully, developers must clearly understand:
  - What a component does
  - What services it provides
  - What services it requires
  - How other components communicate with it
  - What conditions must be satisfied before using it
  - What results should be produced
  - What quality attributes it must satisfy

This leads to three closely related concepts:

- **Components + Interfaces + Specifications = Well-defined building blocks for software systems**
- A component provides functionality, an interface defines how that functionality is accessed, and a specification describes what the component is expected to do.

Software Components

Definition of a Software Component

- A **software component** is an independently deployable and reusable unit of software that provides a well-defined set of services through specified interfaces.
- In simpler terms:
- A software component is a self-contained piece of software designed to perform a particular function and interact with other software through clearly defined interfaces.
- Examples include:
- Authentication component
- Payment-processing component
- Database component

Encapsulation and Modularity

Encapsulation

- **Encapsulation** is the practice of combining data and operations into a unit while restricting direct access to the internal implementation. In CBSE, encapsulation means that users of a component do not need to know how the component performs its operations.
- **Example**
- Suppose we have a **Payment Component**.

A client may call:

processPayment(amount, account)

- The client does not need to know whether the component internally uses:
  - Database queries
  - Encryption
  - Network communication
  - Transaction validation
  - Error handling
  - The implementation remains hidden.

Benefits of encapsulation

- Reduces complexity
- Protects internal data
- Reduces coupling
- Makes maintenance easier
- Supports component reuse
- Allows implementation changes without affecting clients

Modularity

- **Modularity** is the process of dividing a large software system into smaller, manageable and relatively independent units called modules or components.

Example

- A banking application could be divided into:
- Banking System
  - *Customer Management*
  - *Account Management*
  - *Transactions*
  - *Loans*
  - *Payments*
  - *Notifications*
  - *Reporting*
- Each module performs a specific responsibility.

Good modular design

- A good modular system should have:
- **High cohesion** –
- Elements within a component are strongly related.
- A component should focus on one closely related set of responsibilities.

- **Low coupling** –
- Components have minimal unnecessary dependencies.
- A component should not depend heavily on the internal implementation of other components.

Reusability and Replaceability

Reusability: Reusability is the ability of a software component to be used in multiple applications or contexts without requiring substantial redevelopment.

- **Example**
- An **authentication component** may provide: login(), logout(), changePassword(), resetPassword()
- The same component could be used in:
- University systems
- Hospital systems
- Government systems
- E-commerce applications

Factors that improve reusability

- A component is more reusable when it has:
- Clear functionality
- Well-defined interfaces
- Minimal dependencies
- Good documentation
- Configurability
- Standard communication mechanisms
- Good portability
- Proper error handling

Replaceability

- **Replaceability** refers to the ability to replace one component with another component that provides equivalent functionality without requiring major changes to the rest of the system.
- **Importance of replaceability**
  - Prevents vendor lock-in
  - Supports system evolution
  - Simplifies upgrades
  - Allows technology changes
  - Improves maintainability
  - Provides flexibility

Component Granularity

- **Component granularity** refers to the size and scope of functionality contained within a software component.
- Components can be:
- Fine-grained
- Medium-grained
- Coarse-grained

Fine-grained components

- A fine-grained component provides a small amount of functionality.
- **Example**
- A component that performs: CalculateTax()

Advantages

1. Highly reusable
2. Easy to understand
3. Easy to test
4. Can be combined flexibly

Disadvantages

1. Large number of components may be required
2. More communication between components
3. Integration may become complicated

Coarse-grained components

A coarse-grained component provides a relatively large business function.

- **Example**
- A complete **Student Registration Component** might provide:
- Student search
- Course selection
- Registration
- Fee calculation
- Registration confirmation

- **Advantages**
- Fewer components
- Less communication overhead
- Easier to deploy as a business service
- **Disadvantages**
- May be less reusable
- More difficult to modify
- May contain unnecessary functionality for some applications

Choosing appropriate granularity

- There is no universally correct component size.
- Developers should consider:
- Reusability
- Performance
- Complexity
- Communication overhead
- Maintainability
- Business requirements
- Deployment requirements

General principle

- Components should be large enough to provide meaningful functionality but small enough to remain manageable and reusable.

Black-Box and White-Box Components

- **Black-box components:** A **black-box component** is a component whose internal implementation is hidden from the developer using it.
- The developer knows:
- What services it provides
- How to invoke the services
- Required inputs
- Expected outputs
- But does not need to know how it works internally.
- **Example**
- A developer uses an external payment API:
- makePayment(amount, customer)
- The developer does not need access to the payment provider's internal source code.

Cot:

Advantages

- Strong encapsulation
- Easy reuse
- Reduced complexity
- Implementation can change independently
- Promotes loose coupling

Disadvantages

- Limited customization
- Difficult to modify internal behaviour
- Internal defects may be difficult to diagnose
- Dependency on component provider

White-Box Components

- A **white-box component** is a component whose internal implementation is visible and potentially modifiable by the developer.
- The developer may have access to:
- Source code
- Internal classes
- Algorithms
- Data structures
- Configuration
- **Example:** An organization downloads an open-source reporting library, examines its source code and modifies it to meet specific requirements.

Cont:

- **Advantages**
- Greater customization
- Internal behaviour can be modified
- Easier to investigate some problems
- Can be optimized for a specific application
- **Disadvantages**
- Greater maintenance responsibility
- Higher risk of modifying the component incorrectly
- More knowledge of internal implementation is required
- Modifications may make future upgrades difficult

Component Interfaces

Definition

- A **component interface** is a defined set of operations or services through which a component communicates with other components or clients.
- The interface describes **what the component can do**, without necessarily describing **how it does it.**
- **Example**
  - A Student component may expose:
  - registerStudent()
  - updateStudent()
  - findStudent()
  - deleteStudent()
  - Other components use these services through the interface.

2 Importance of interfaces

- Interfaces:
- Promote encapsulation
- Reduce coupling
- Support interoperability
- Enable component replacement
- Define communication rules
- Make components easier to reuse
- Separate implementation from usage

Provided and Required Interfaces

- A component can have two important types of interfaces:

1 Provided interface

- A **provided interface** describes services that a component makes available to other components.

2 Required interface

- A **required interface** describes services that a component needs from other components in order to perform its own functionality.

Example

- Think of a restaurant.
- **Provided services:** meals and drinks the restaurant offers.
- **Required services:** electricity, water, food supplies and payment services that the restaurant needs.
- A software component similarly **provides services** while **requiring services** from its environment.

Component Contracts

- **Definition:** A **component contract** is a formal or informal agreement that specifies the obligations, expectations and behaviour associated with a software component and its interactions with other components.
- A contract answers questions such as:
- What does the component provide?
- What inputs are acceptable?
- What output should be produced?
- What conditions must hold before an operation?
- What conditions will hold after an operation?
- What errors can occur?
- What quality requirements apply?

Design by Contract

- Component contracts are closely associated with **Design by Contract (DbC)**.
- A contract normally consists of:
- **1. Preconditions :**Conditions that must be true **before** an operation executes.
- **2. Postconditions:** Conditions that must be true **after** successful execution.
- **3. Invariants:** Conditions that must remain true throughout the lifetime or valid operation of a component.

Preconditions

- **Definition :**A **precondition** is a condition that must be satisfied before a component operation can be executed correctly.
- **Example**
- Consider: withdraw(amount), Possible preconditions:, Account must exist., Account must be active., Amount must be greater than zero. , Sufficient balance must be available.
- Therefore: Preconditions:
- 1. amount > 0
- 2. account exists
- 3. balance >= amount
- If these conditions are not satisfied, the operation may fail or reject the request.

Postconditions

- **Definition:** A **postcondition** is a condition that should be true after an operation has completed successfully.
- For the withdrawal operation:
- withdraw(5000)
- Possible postconditions:
- New balance = Old balance - 5000
- Transaction is recorded
- Withdrawal receipt is generated
- **Example**
- If the account initially has KSh 20,000:
- Initial balance = KSh 20,000
- Withdrawal = KSh 5,000
- Final balance = KSh 15,000
- The postcondition specifies what should be true after successful execution.

Functional Specifications

- **Definition:** A **functional specification** describes **what a software component is expected to do.**
  It identifies: Services provided, Inputs Outputs Processing behaviour, Business rules and Error conditions

Example: Student Registration Component

- Functional requirements may specify that the component must:
- Register a new student.
- Assign a unique student number.
- Allow students to select courses.
- Check course prerequisites.
- Record registration.
- Generate registration confirmation.
- These describe **what the component does.**

Non-Functional Specifications

- **Definition:** A **non-functional specification** describes the quality attributes, constraints and operational characteristics of a component rather than its specific business functions.
- Common non-functional requirements include:
- Performance, Security, Reliability, Availability, Scalability, Usability
- Maintainability
- Portability
- Interoperability
- **22.2 Examples**
- For a payment component:

Example

Functional specification

- The component shall process a payment and return a transaction confirmation.

Non-functional specifications

- The component shall:
- Process 95% of valid requests within 2 seconds.
- Encrypt sensitive information during transmission.
- Maintain an availability of 99.9%.
- Support 1,000 concurrent requests.
- Record all successful transactions for auditing.
- The first statement describes **what it does**; the others describe **how well or under what constraints it must operate**.

Component Specification

- A complete component specification should provide enough information for another developer to determine whether the component can be used.
- A component specification may include:

1. Component name: Example: Student Registration Component
2. Purpose: What problem does the component solve?
3. Provided interfaces: Services offered.
4. Required interfaces: Services required from other components.
5. Functional specification: What the component does.

Cont.: Component Specification

6. Non-functional specification: Performance, security, reliability and other constraints.

7. Preconditions: Conditions that must be satisfied before operations.

8. Postconditions: Expected results after operations.

9. Dependencies: Other software, libraries, databases or services required.

10. Error conditions: Possible failures and how they are reported.

11. Configuration requirements: Settings that must be provided.

12. Version and compatibility information: Supported platforms, versions and environments

Example of a Complete Component Specification

- **Component: Student Registration Component**
- **Purpose:** Provides services for registering students for academic programmes and courses.
- **Provided interface** ***registerStudent()*****,** ***updateStudent()*****,** ***findStudent()*****,** ***registerCourse()*****,** ***cancelCourse()***
- **Required interface**
- ***StudentDatabase*****,** ***AuthenticationService*****,** ***CourseManagementService*****,** ***PaymentService***

Functional specifications

The component shall:

- Create a student registration record.
- Validate student information.
- Check course availability.
- Verify prerequisites.
- Record selected courses.
- Generate registration confirmation.

Non-functional specifications

- The component shall:
- Protect student information.
- Provide acceptable response times.
- Support concurrent users.
- Maintain transaction integrity.
- Be available during registration periods.

Importance of Interfaces in CBSE

- Interfaces are particularly important because they allow components to interact without requiring knowledge of their internal implementation.
- **Without interfaces**
- Component A ——————► Internal code of Component B
- This creates strong coupling.
- **With interfaces**
- Component A ——————► Interface B ——————► Component B
- This promotes:

Loose coupling, Encapsulation, Replaceability, Reusability, Maintainability

Interface Compatibility

- For two components to work together, their interfaces must be compatible.
- Compatibility may involve:

Data compatibility

- Both components agree on:
  - Data types
  - Formats
  - Units
  - Encoding

Operation compatibility

- The required operation must exist in the provided interface.

Protocol compatibility

- Components must agree on communication mechanisms.

Behavioural compatibility

The component must behave according to the expectations of its client.

Interface Mismatch

- An **interface mismatch** occurs when a required interface does not match the interface provided by another component.
- **Example**
- Component A requires:
- getCustomerName(customerID)
- Component B provides:
- findCustomer(customerNumber)
- Although the functionality may be similar, the interfaces do not directly match

Adapters and Wrappers

- An **adapter** is a component or mechanism that converts one interface into another compatible interface.
- **Example**
- Suppose an application expects:
- sendSMS(phone, message)
- but an existing SMS service provides:
- transmit(number, text)
- An adapter can translate between the two interfaces.
- This allows the existing component to be reused without modifying its internal implementation.

Good Practices in Component Specification

1. When specifying a component, developers should:
2. Clearly define its purpose.
3. Specify all provided interfaces.
4. Specify all required interfaces.
5. Define inputs and outputs.
6. Document preconditions.
7. Document postconditions.
9. Specify functional behaviour.
10. Specify non-functional requirements.
11. Document dependencies.
12. Define possible error conditions.
13. State compatibility requirements.
14. Keep specifications precise and unambiguous.
15. Update documentation when the component changes.

References

1. Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.
2. Szyperski, C., Gruntz, D., & Murer, S. (2002). *Component Software: Beyond Object-Oriented Programming* (2nd ed.). Addison-Wesley Professional.
3. Heineman, G. T., & Councill, W. T. (Eds.). (2001). *Component-Based Software Engineering: Putting the Pieces Together*. Addison-Wesley.

End

!!!!!!!